СЕССИОННОЕ ЗАДАНИЕ
по дисциплине «Основы криптографии»

1. Трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны

Трехсторонний протокол распределения ключей нужен, чтобы два участника могли получить общий ключ для защищенного обмена данными с помощью доверенного сервера. В качестве примера рассмотрим протокол Нидхема — Шрёдера с симметричным шифрованием.

В протоколе участвуют три стороны. Участники A и B хотят обмениваться данными, а сервер S выполняет роль центра распределения ключей (KDC). До начала протокола у A есть общий секретный ключ K_{AS} с сервером S, а у B — ключ K_{BS} с тем же сервером. Эти ключи известны только соответствующим участникам и серверу.

1) Участник A запрашивает у сервера S сеансовый ключ для связи с B. Для проверки актуальности ответа A добавляет случайное число N_A:

$$A \rightarrow S: A, B, N_A$$

2) Сервер S генерирует сеансовый ключ K_{AB} и отправляет A сообщение, зашифрованное ключом K_{AS}. Внутри сообщения находятся случайное число N_A, идентификатор B, сеансовый ключ и отдельный блок для B:

$$S \rightarrow A: E_{K_{AS}}(N_A, B, K_{AB}, T_B)$$

$$T_B = E_{K_{BS}}(K_{AB}, A)$$

Обозначение $E_K(M)$ означает шифрование сообщения M ключом K. Участник A расшифровывает свою часть сообщения, проверяет N_A и получает K_{AB}. Блок T_B он не расшифровывает, а передает участнику B.

3) Участник A пересылает блок, предназначенный для B:

$$A \rightarrow B: T_B$$

Участник B расшифровывает его своим ключом K_{BS} и получает сеансовый ключ K_{AB} и идентификатор A.

4) Для дополнительной проверки владения ключом B может отправить A случайное число N_B, зашифрованное ключом K_{AB}. A расшифровывает его и возвращает, например, значение $N_B - 1$, также зашифрованное K_{AB}:

$$B \rightarrow A: E_{K_{AB}}(N_B)$$

$$A \rightarrow B: E_{K_{AB}}(N_B - 1)$$

Если B правильно проверяет ответ, это подтверждает, что A также владеет сеансовым ключом.

В результате A и B получают общий сеансовый ключ K_{AB} и могут использовать его для симметричного шифрования сообщений. Сервер S участвует в распределении ключа, но не обязан участвовать в дальнейшем обмене данными. У базового протокола Нидхема — Шрёдера есть уязвимости, связанные с повторным воспроизведением старых сообщений при определенных условиях, поэтому реальные протоколы используют дополнительные проверки свежести и срока действия ключей.

2. Формальные модели шифров

Формальная модель шифра — это математическое описание того, как открытый текст преобразуется в шифротекст и как затем восстанавливается исходное сообщение. В модели задаются множества сообщений, ключей и шифротекстов, а также алгоритмы шифрования и расшифрования.

2.1. Симметричный шифр

В симметричном шифре для шифрования и расшифрования используется один секретный ключ. Обозначим через M множество открытых текстов, C — множество шифротекстов, K — множество ключей. Тогда шифрование и расшифрование задаются функциями:

$$E: K \times M \rightarrow C$$

$$D: K \times C \rightarrow M$$

Для любого допустимого сообщения m и ключа k должно выполняться условие корректности:

$$D_k(E_k(m)) = m$$

То есть после расшифрования должен получаться исходный открытый текст. Важную роль играет генерация ключа: он должен быть достаточно случайным и иметь достаточную длину, чтобы затруднить подбор.

2.2. Асимметричный шифр

В асимметричной системе используются два связанных ключа: открытый pk и закрытый sk. Открытый ключ можно передавать другим пользователям, а закрытый должен храниться в секрете. Пара ключей формируется алгоритмом генерации:

$$(pk, sk) \leftarrow Gen(1^{\lambda})$$

Здесь λ — параметр безопасности. Шифрование выполняется открытым ключом, а расшифрование — закрытым:

$$c = E_{pk}(m)$$

$$m = D_{sk}(c)$$

Условие корректности имеет вид:

$$D_{sk}(E_{pk}(m)) = m$$

Такая схема позволяет зашифровать сообщение для владельца закрытого ключа, не передавая отправителю секретный ключ. При этом важно удостовериться, что открытый ключ действительно принадлежит нужному человеку или системе, например с помощью сертификата.

2.3. Вероятностный шифр

В вероятностном шифре при шифровании используются случайные данные. Поэтому одно и то же сообщение при одном ключе может каждый раз превращаться в разные шифротексты. Шифрование можно записать так:

$$c \leftarrow E_k(m;\, r)$$

Здесь r — случайное значение. Расшифрование выполняется с помощью ключа:

$$m = D_k(c)$$

Корректность означает, что для допустимых сообщений и случайных значений расшифрование возвращает исходный текст: $D_k(E_k(m;\, r)) = m$. Случайность помогает скрывать закономерности, например не позволяет по одному только совпадению шифротекстов легко определить, что сообщения одинаковые.

2.4. Модель цифровой подписи

В криптографических системах также используется цифровая подпись. Она не предназначена для сокрытия сообщения, а помогает проверить его целостность и подлинность отправителя. Сначала создается пара ключей, затем подпись формируется закрытым ключом, а проверяется открытым:

$$(sk, pk) \leftarrow Gen(1^{\lambda})$$

$$\sigma \leftarrow Sign_{sk}(m)$$

$$Verify_{pk}(m, \sigma) \in \{0, 1\}$$

Если проверка возвращает 1, подпись считается корректной для данного сообщения и открытого ключа. Таким образом, формальные модели описывают состав криптографической системы, назначение ее алгоритмов и условия правильной работы.

3. Принципы построения поточных систем шифрования

Поточное шифрование обрабатывает данные последовательно — например, по одному биту или небольшими группами битов. При этом формируется ключевая последовательность, или гамма, которая объединяется с открытым текстом. Часто для этого используется операция XOR:

$$C = M \oplus G$$

$$M = C \oplus G$$

Здесь M — открытый текст, C — шифротекст, G — гамма, а $\oplus$ — побитовое исключающее ИЛИ. Повторное применение XOR с той же гаммой возвращает исходные данные.

При построении поточных шифров важно соблюдать несколько принципов.

1) Стойкость гаммы. Гамма должна быть непредсказуемой для противника. Если в ней есть заметные закономерности, они могут помочь восстановить открытый текст. Поэтому используют криптографически стойкие генераторы или специальные алгоритмы формирования гаммы.

2) Надежность ключа. Секретный ключ должен быть достаточно длинным, случайным и защищенным от раскрытия. Если ключ короткий или предсказуемый, его можно подобрать перебором.

3) Недопустимость повторного использования гаммы. При использовании одной и той же гаммы для двух сообщений получаем $C_1 = M_1 \oplus G$ и $C_2 =$

$M_2 \oplus G$. Тогда $C_1 \oplus C_2 = M_1 \oplus M_2$. Это раскрывает связь между сообщениями и может помочь противнику восстановить их содержимое.

4) Правильная синхронизация. Отправитель и получатель должны формировать одну и ту же гамму для соответствующих частей сообщения. Для этого используются согласованные начальные состояния, счетчики или другие параметры. Если синхронизация нарушается, расшифрование может стать неправильным.

5) Эффективность. Поточные шифры должны быстро обрабатывать данные и подходить для передачи потоков информации. Во многих случаях они не требуют дополнения сообщения до размера полного блока, что удобно для сообщений произвольной длины.

6) Контроль ошибок. В обычном синхронном поточном шифре ошибка в одном бите шифротекста обычно меняет соответствующий бит открытого текста. В самосинхронизирующихся схемах ошибка может повлиять и на следующие символы, пока система не восстановит синхронизацию.

7) Контроль целостности. Само шифрование не всегда защищает от намеренного изменения сообщения. Поэтому для обнаружения подмены используют код аутентификации сообщения (MAC) или режимы, которые одновременно обеспечивают конфиденциальность и целостность.

Итак, хорошая поточная система должна обеспечивать стойкую гамму, надежное управление ключами, отсутствие повторов гаммы, согласованную синхронизацию и достаточную скорость работы.

4. Режим шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования

В качестве примера можно рассмотреть режим шифрования с обратной связью по шифртексту — CFB (Cipher Feedback). В этом режиме блочный шифр используется для получения гаммы, а результат зависит от предыдущих сегментов шифротекста.

Пусть R_i — регистр обратной связи на шаге i, P_i — текущий сегмент открытого текста, C_i — соответствующий сегмент шифротекста, а s — длина сегмента. Тогда шифрование задается формулой:

$$C_i = P_i \oplus MSB_s(E_k(R_i))$$

Функция MSB_s берет первые s бит результата шифрования. После этого регистр сдвигается, и в него добавляется очередной сегмент шифротекста.

Расшифрование выполняется так:

$$P_i = C_i \oplus MSB_s(E_k(R_i))$$

Важная особенность CFB заключается в том, что при расшифровании используется прямое шифрующее преобразование E_k , а не обратное преобразование D_k. Получатель воспроизводит тот же регистр обратной связи, вычисляет гамму и объединяет ее с полученным сегментом шифротекста операцией XOR. Поэтому для такого механизма не требуется обращать базовое отображение на каждом шаге.

При этом в стандартной реализации CFB базовый блочный шифр обычно сам является биективным преобразованием блоков. Пример показывает более узкое утверждение: для однозначного расшифрования в CFB не требуется использовать обратимость базового преобразования как отдельный шаг расшифрования. Достаточно согласованно вычислять гамму и выполнять XOR с

соответствующим сегментом шифротекста. Поэтому корректность режима связана с механизмом обратной связи и обратимостью XOR, а не с обращением каждого результата E_k.

5. Сколько 64-битовых блоков открытого текста искажается при ошибке одного бита шифротекста в режиме CFB

По условию длина блока базового шифра равна 64 битам, а длина сегмента открытого и шифрованного текста — 8 битам. В шифротексте изменился один бит. Нужно определить, сколько 64-битовых блоков открытого текста может быть затронуто при расшифровании.

В режиме CFB очередной сегмент открытого текста получается путем XOR сегмента шифротекста с частью результата шифрования регистра обратной связи. Поэтому ошибка в текущем сегменте непосредственно меняет соответствующий бит открытого текста. Кроме того, ошибочный бит попадает в регистр и влияет на формирование гаммы для следующих сегментов.

Длина регистра равна 64 битам, а один сегмент занимает 8 бит. Следовательно, регистр содержит:

$$64 / 8 = 8 \text{ сегментов}$$

Ошибочный бит непосредственно искажает один 8-битовый сегмент. Затем он остается в регистре обратной связи и влияет на последующие сегменты, пока не выйдет из регистра. Для принятого в задаче подсчета затронуты текущий сегмент и еще восемь сегментов:

$$\mathrm{N} = 1 + 64 / 8 = 9 \text{ сегментов}$$

Это соответствует 9 × 8 = 72 битам открытого текста. Один 64-битовый блок состоит из восьми сегментов по 8 бит. Поэтому 9 последовательных сегментов могут пересекать границу двух 64-битовых блоков:

$$72 / 64 = 1{,}125 \text{ блока}$$

Следовательно, могут быть затронуты два 64-битовых блока открытого текста. В первом сегменте меняется соответствующий бит, а в последующих сегментах из-за изменения регистра обратной связи могут искажаться другие биты. После выхода ошибочного бита из регистра расшифрование следующих сегментов восстанавливается, если новых ошибок нет.

Ответ: 2 блока по 64 бита.

При этом важно не путать размер блока базового шифра (64 бита) с размером сегмента CFB (8 бит).

6. Минимальная длина хэш-значения для вероятности коллизии не более 10^{-9}

Коллизия хэш-функции возникает, когда два разных сообщения имеют одинаковое хэш-значение: $m_1 \neq m_2$, но $H(m_1) = H(m_2)$. По условию хэш-функция возвращает двоичный вектор длины n. Значит, существует 2^n возможных хэш-значений.

Предположим, что хэш-значения двух случайных сообщений независимы и равномерно распределены. После выбора первого значения вероятность того, что второе совпадет с ним, равна:

$$P(\text{коллизии}) = 1 / 2^n$$

По условию эта вероятность не должна превышать 10^{-9}. Получаем неравенство:

$$1 / 2^n \leq 10^{-9}$$

Перенесем выражение, чтобы получить:

$$2^n \geq 10^9$$

Применим двоичный логарифм к обеим частям:

$$n \geq log_2(10^9)$$

Используя свойство логарифма степени, получаем:

$$n \geq 9\, log_2(10)$$

Так как $log_2(10) \approx 3{,}321928$, то:

$$n \geq 9 \times 3{,}321928 = 29{,}897352$$

Размерность n должна быть целым числом, поэтому округляем полученную границу вверх:

$$n_{min} = \lceil 29{,}897352 \rceil = 30$$

Проверим результат. При n = 30:

$$P(\text{коллизии}) = 1 / 2^{30} = 1 / 1\,073\,741\,824$$

$$P(\text{коллизии}) \approx 9{,}3132 \times 10^{-10}$$

Полученное значение меньше 10^{-9}, то есть условие выполняется. Для сравнения, при n = 29:

$$P(\text{коллизии}) = 1 / 2^{29} \approx 1{,}8626 \times 10^{-9}$$

Это значение больше допустимой вероятности, поэтому 29 бит недостаточно.

Ответ: минимальная длина хэш-значения равна 30 битам.

В этом решении рассматривается вероятность совпадения хэш-значений именно у одной случайной пары сообщений. Если оценивается вероятность хотя бы одной коллизии среди большого числа сообщений, применяется модель «парадокса дней рождений», и необходимая длина хэш-значения будет больше.